

Assignment 1: Process, Threads, IPC

Suhaib Abdi Muhummed, muhummed@kth.se

Abdulaziz Hassan Farah, ahfarah@kth.se

Daniel Sherif, dsheerif@kth.se

Question 1: Basic information of your OS environment.

What are the main OS distributions today? What do you think are the relationship of an OS and the kernel? Please name three main tasks that Operating Systems do.

Specify the Linux environment you choose to use for this course. Which OS distribution and kernel version do you have?

Please specify the commands you used in [2] for checking the information.

Answers:

Main OS distribution today are windows, macOS and linux.

The relationship between OS and Kernel is that the OS consists of three main parts: user interface, system call and kernel. The user interface is what users see such as graphical and system calls are how the user programs request access to something from the kernel service. And the kernel is a layer between the user space and hardware. The kernel is the heart of OS, it manages the communication between software (user space) and hardware. One of the main tasks that operating systems do is managing resources. This means that the OS manages all resources from hardware such as CPU, memory, disk, network, I/O system, and monitor.

The second main task that OS does is controlling programs. The OS controls the execution of programs so that we can avoid errors that can come from wrong use of the hardware.

The third main task that operating systems do is providing user interface, which allows the user to interact with the computer through commands or graphical screens.

For this course we use ubuntu with version 20.04.6 LTS (Focal Fossa) and for the kernel version: Linux 5.4.0-222-generic.

Commands: for OS: `cat /etc/os-release` and for kernel: `hostnamectl`. It also shows the OS version.

Question 2: Process Memory Layout.

Analyze the following code and consider the case where this program is executed by a process. Describe the expected memory layout and explain the parts that will be stored in the text, data, heap, and stack section, respectively. Your analysis should include all variables.

Answer:

In memory layout divided first to block text and data where then the block of data can be divided by data static and dynamic data which then even can divide in data static into uninitialized and initialized and for dynamic data it also divides to stack and heap.

Stack: here in memory all local variables have been stored. For example in this code those variables are those two (i) from `getSum` and `main` and `s`.

Heap: when we want to allocate some memory for a specific task for example large array size or when we aren't sure how large size may user want then we can allocate some memory in heap by using `malloc` in C and then make sure to free that memory by using function `free` in C. For example in the code the variable (`a`) will be stored in heap.

Uninitialized: here when we make global variable that we don't initialize by some value then it will be stored in uninitialized, example in code the variable `sum` and pointer `a` will be stored in uninitialized part of data segment.

Initialized: when we make a global variable which we then initialize by some value then this variable goes in memory in the initialized part in static. In the code `N` will be stored in initialized part of data segment.

Text segment: here goes all functions we make in this part of the memory text segment we all can read in the code here the function `main` and `getSum` will be stored.

```
4  #define N 1024
5
6  double sum;
7  double *a;
8
9  double getSum ( double *x ) {
10     int i;
11     double s = 0 ;
12
13     for ( i = 0 ; i < N; i ++ )
14         s += x[i] ;
15     return s;
16 }
17
18 int main () {
19     int i;
20     a = (double*) malloc(N*sizeof(double));
21
22     for ( i = 0; i < N; i ++ ) a[i] = i;
23
24     sum = getSum(a);
25     printf( "sum = %.2f\n", sum );
26     return 0;
27 }
```

Figure 1: Memory layout illustration

Question 3: Process Creation.

1. How many processes will be created by the following code (including the parent process)? The total number of processes created (including the original parent) is $2^N * 2^N = 2^{2N}$. Since $N=3$, that means $2^{2*3} = 2^6 = 64$ processes.
2. How will you verify the number of processes created in this program? By printing the process IDs.
3. How many processes are created as a function of N ? Total Processes = 2^{2N} .

```
4  #define N 3
5
6  int main () {
7
8      for (int i=0;i<N;i++)
9      {
10         fork();
11         fork();
12     }
13
14     return 0;
15 }
```

Figure 2: Fork function illustration

Question 4: Sharing among processes and threads

1. Which of the following may be shared between the parent and child process after forking? Explain your answer, including the definition of 'shared' in your context.
 - (a) **Program:** Both parent and child processes execute the same program code so they share. They typically refer to the same read-only code segment in memory.
 - (b) **Process ID:** The Process ID (PID) is a unique identifier. The child process receives a new, unique PID from the operating system, while the parent retains its original PID.
 - (c) **File descriptors:** The child receives a copy of the parent's file descriptor table. Crucially, these copied descriptors refer to the same underlying open file object in the kernel. This means they share the same file pointer (offset) and flags.
 - (d) **Program counter:** The Program Counter (PC) is a CPU register that tracks the next instruction. It is part of the process's CPU context. While the PC value is the same immediately after the fork (as the child starts executing at the same point), it is not shared memory. Each process has its own set of registers and its PC will diverge immediately upon execution.

2. Which of the following may be shared among threads in a multi-threaded process? Explain your answer, including the definition of 'shared' in your context.
- (a) **Registers:** Each thread has its own private set of CPU registers, including the Program Counter (PC), stack pointer (SP), and general-purpose registers. This is essential for the OS to suspend one thread and resume another. If they shared registers, they would overwrite each other's execution state and could not run independently. Not Shared.
 - (b) **Heap:** The heap is a global region of the process's address space. Any thread can call `malloc()` or `new` to allocate memory, and a pointer to that memory can be passed to and manipulated by any other thread. This is a primary method of direct communication between threads, and changes to heap data are visible to all. Yes Shared.
 - (c) **Global variables:** Global (and static) variables are allocated in the data segment of the process, which is part of the shared address space. All threads have equal access to the same memory locations for these variables. Modifying a global variable in one thread immediately changes its value for all other threads in the process. Yes Shared
 - (d) **Stack:** Each thread is created with its own private stack. This is necessary to track its unique sequence of function calls, local variables, and return addresses. The threads *can* access each other's stacks if given a pointer (since they share the same virtual address space), but the stack themselves are independent function execution.

Question 5: Sharing among processes and threads

What will be the printout of "CHILD: value=? Explain your answer.

What will be the printout of "PARENT: value=? Explain your answer.

```
6  #define N 5
7
8  int value = 0;
9
10 void *thread_func(void *param);
11
12 int main(int argc, char *argv[])
13 {
14     pid_t pid;
15     pthread_t tid;
16     pthread_attr_t attr;
17
18     pid = fork();
19
20     if (pid == 0) { /* child process */
21         pthread_attr_init(&attr);
22         for(int i=0; i<N; i++){
23             pthread_create(&tid,&attr,thread_func,NULL);
24             pthread_join(tid,NULL);
25         }
26         printf("CHILD: value = %d\n",value);
27     }
28     else if (pid > 0) { /* parent process */
29         wait(NULL);
30         printf("PARENT: value = %d\n",value);
31     }
32 }
33
34 void *thread_func(void *param) {
35     value ++;
36     pthread_exit(0);
37 }
```

Figure 3: Fork function illustration

1. The printout of the child value will be **5**, because `N` is defined as 5 and five threads are created. For every thread, the variable `value` increments; hence for every created thread in the child process, the value is incremented by 1. Because there are five threads created, the printout is also 5.
2. Threads are only created in the child, which is a separate process from the parent. This means they are not directly related, and the information in each process does not affect the other. After using `fork()`, the process memory is no longer shared. This means that parent and child processes have two different versions of the global variable `value`. For the parent, this remains 0, as it never gets changed, while for the child, it is exclusively changed to 5 due to the threads running five times only in the child process.

Therefore, the printout for the parent remains **0**, as it is unchanged.

Question 6: Inter-process Communication.

Analyze the following codes for using FIFO for IPC. Identify Producer and Consumer in these two codes. Analyze the time dependency among the Producer and Consumer processes. Draw a timeline to illustrate the main function calls including mkfifo, open, readwrite, close, unlink, print, in the producer and consumer processes.

Code Block A

Code Block B

```
1  #include <stdio.h>
2  #include <unistd.h>
3  #include <fcntl.h>
4  #include <sys/stat.h>
5  #include <string.h>
6
7  #define BUFSIZE 1024
8
9  int main () {
10
11     const char *name = "/tmp/myfifo";
12     char recv_buf[BUFSIZE];
13
14     int fd;
15
16     mkfifo(name, 0666);
17
18     sleep(10);
19
20     fd = open(name, O_RDONLY);
21     read(fd, recv_buf, BUFSIZE);
22     close(fd);
23
24     printf("Received %s \n", recv_buf);
25     return 0;
26 }
```

```
1  #include <stdio.h>
2  #include <unistd.h>
3  #include <fcntl.h>
4  #include <sys/stat.h>
5  #include <string.h>
6
7  int main () {
8
9     const char *name = "/tmp/myfifo";
10    const char *msg = "hello";
11    int fd;
12
13    mkfifo(name, 0666);
14
15    fd = open(name, O_WRONLY);
16    write(fd, msg, strlen(msg)+1);
17    close(fd);
18
19    unlink(name);
20
21    return 0;
22 }
```

Figure 4: Code block

1. Code block A is the consumer, as it reads from the FIFO.
Code block B is the producer, as it is the one writing.
2. Timeline:

Consumer	Producer
<code>mkfifo(name, 0666) // creates fifo</code>	<code>mkfifo(name, 0666) // creates fifo</code>
<code>sleep(10) // wait for producer</code>	–
–	<code>open(name, O_WRONLY) // block until reader opens</code>
<code>open(name, O_RDONLY) // reader open, unblock both</code>	–
<code>read() // receives data</code>	<code>write() // sends data ("hello")</code>
<code>close(fd)</code>	<code>close(fd)</code>
<code>printf("%s", msg)</code>	<code>unlink(name)</code>

Programming Section:

Question 7: Understand the different statuses of a process and their transition.

Analyze this code that calls `fork()` to create a child process.

```
1  #include <stdio.h>
2  #include <unistd.h>
3
4  int main() {
5
6      int pid = fork();
7      if(pid == 0)
8          printf("I'm the child %d\n", getpid());
9      else{
10         printf("My child is called %d\n", pid);
11         sleep(10);
12     }
13
14     printf("PID%d says goodbye\n", getpid());
15     return 0;
16 }
```

Figure 5: statuses of Process.

Please explain the purpose of inspecting the returned value from `fork()`.

- The return value from `fork()` is crucial because it tells the executing code which process it is running in:
 - **Return `pid == 0`:** This means we are in the child process.
 - **Return `pid > 0`:** This means we are in the parent process, and `pid` is the process ID of the child.
 - **Return `pid < 0`:** This means `fork` failed.

This is the primary mechanism for creating divergent execution paths. Both processes start executing from the same point after `fork()`, but based on the return value, they can take different actions.

2. Explain what a zombie process and orphan process mean. In the code in [1], do you see any zombie and orphan process? Explain why.

- A zombie process is a process that has completed execution (terminated) but still has an entry in the process table because its parent hasn't yet called `wait()` to read its exit status.

Characteristics:

- Process has finished execution
- Still consumes a PID and process table entry
- Resources are freed, but exit status is preserved
- Remains until parent collects exit status via `wait()`

- An orphan process is a process whose parent has terminated. Orphan processes are automatically adopted by the `init` process (PID 1).

Characteristics:

- Parent dies before child
- Child continues running normally
- `init` becomes the new parent
- No negative consequences - properly cleaned up when child terminates

3. Run the code and check the status of the processes using the `ps` command. What status do you see for the parent process? What status do you see for the child process? You may include a screenshot in the report.

- The parent calls `sleep(10)` after forking. The child process will:
 1. Print its message
 2. Print "says goodbye"
 3. Terminate immediately

Output:

```
My child is called 51107
I'm the child 51107
PID51107 says goodbye
PID51106 says goodbye
```

Process Status:

```
51106    42802    0 19:01 pts/1    00:00:00 ./Task7
51218    46873    0 19:01 pts/3    00:00:00 grep --color=auto Task7
51233    46873    0 19:01 pts/3    00:00:00 grep --color=auto Task7
```

Execution Timeline:

```
Time: 0s      Parent: forks creates child 48002
Parent: prints "My child is called 48002"
Parent: goes to sleep for 10 seconds
Time: ~0.1s   Child: prints "I'm the child 48002"
Child: prints "PID48002 says goodbye"
Child: EXITS becomes ZOMBIE
Time: 0s-10s  Parent: Sleeping (state S)
Child: Zombie (state Z, shows as <defunct>)
Time: 10s     Parent: wakes up, prints "PID48001 says goodbye"
Parent: exits zombie child gets reaped
```

During this 10-second sleep period, the terminated child becomes a zombie process. The parent outlives the child. The parent sleeps for 10 seconds while the child terminates almost immediately. Since the parent dies after the child, the child never becomes an orphan.

Question 8: Process creation and termination. This is a programming assignment. Write a C program based on the `fork()` system call. The program will create 2 child processes and perform the following tasks.

- The parent process will create an array of N random numbers whose values are in the range of $[0,1]$. N is passed by the user via function arguments.
- Child 1 will find the sum of the first half of the array. Child 2 will find the sum of the second half of the array.
- The parent will print the sum of the whole array using the results returned by the two child processes.
- Add a timer to time from the process creation till parent printing the final result.
- Submit your codes and Makefile in a zip file.
- In your report, briefly explain your design and implementation for each step.
- Run your program with $N=100, 1000, 10000, 100000, 1000000$. Report the timing in a bar chart. Analyze the performance results and correctness.

The code solution can be found on GitHub: Task8.

Correctness Verification

The results from the figure 6 below show excellent correctness and statistical consistency with theoretical expectations. For random numbers uniformly distributed in $[0, 1]$, the expected sum is $N/2$. Observed sums (53.85, 489.46, 4956.00, 49798.30, and 499496.07 for $N = 10^2, \dots, 10^6$) yield percent errors of 7.70%, 2.11%, 0.88%, 0.40%, and 0.10% respectively, showing rapid convergence toward the expected value as N increases. These results confirm that the program's random number generation, partial-sum computation by child processes, and result aggregation by the parent process all function correctly.

N	Observed sum	Expected (=N/2)	Absolute error	Percent error	$\sigma_sum = \sqrt{(N/12)}$	z-score
100	53.85	50.00	+3.85	+7.70%	2.8868	+1.334
1,000	489.46	500.00	-10.54	-2.11%	9.1287	-1.155
10,000	4,956.00	5,000.00	-44.00	-0.88%	28.8675	-1.524
100,000	49,798.30	50,000.00	-201.70	-0.403%	91.2871	-2.210
1,000,000	499,496.07	500,000.00	-503.93	-0.101%	288.6751	-1.746

Figure 6: Observed sum VS Expected sum

Performance Analysis

The performance table 7 and chart 8 together show that the total execution time can be expressed as:

Total time=Overhead (fork + pipe + wait)+Computation (sum of both halves)

For small array sizes, each child only spends around 0.015 to 0.25 ms doing computation, which means that almost the entire total time around 1.3 to 1.5 ms, as shown in the table, is dominated by system overhead rather than useful work.

N	Child Time (ms)	Total Time (ms)	Parallel Efficiency (%)
10	0.015	1.408	0.53
100	0.016	1.523	0.53
1,000	0.018	1.323	0.68
10,000	0.047	1.476	1.59
100,000	0.256	1.329	9.64
1,000,000	1.006	1.965	25.60
10,000,000	9.720	17.002	57.17

Figure 7: Parallel Performance Metrics for Different Array Sizes

This is clearly illustrated in the chart 8 by the large vertical gap between the child-time curve and the total-time curve. As N increases to 1,000,000 and then to 10,000,000, the computation time becomes much more significant, with each child taking 1.006 ms and 9.720 ms respectively, reducing the relative impact of the overhead. In the chart 8, this shift appears as the child-time line gradually approaching the total-time line, showing that computation is finally dominating the runtime. This behavior occurs because overhead remains nearly constant while computation scales roughly linearly with array size, allowing larger workloads to amortize the fixed cost of process creation and communication.

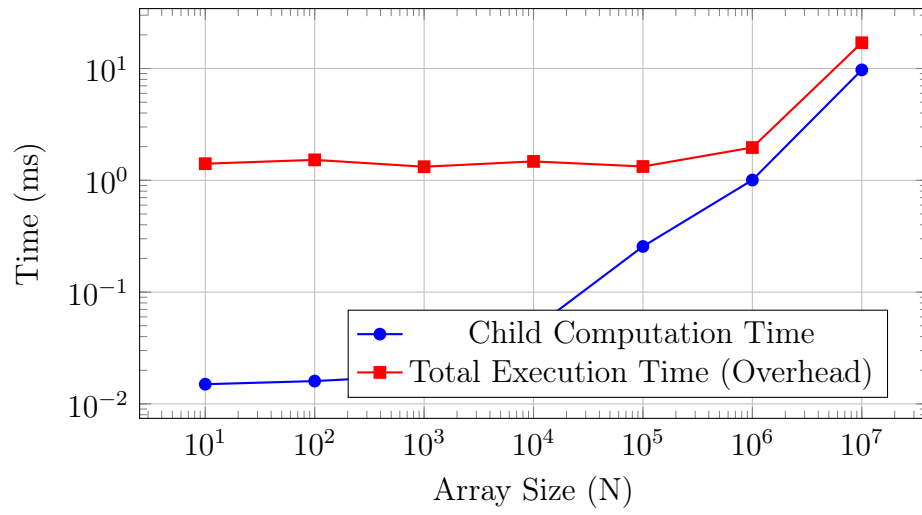


Figure 8: Overhead vs Computation